Getting Started with Python

DATA SCIENCE BOOTCAMP

NASHVILLE SOFTWARE SCHOOL

Objectives

1. Learn the basics about Anaconda and Jupyter Notebook
2. Learn the difference between markdown and code cells
3. Understand file structure setup for working with Jupyter Notebook
4. Learn basic keyboard shortcuts for Jupyter Notebook
5. Gain familiarity with commonly used resources for Python

What is Anaconda?

Anaconda is a popular, open source platform for coding with Python. It is very commonly used by those who code with Python for data analytics and data science.

It includes multiple applications. We will primarily use:

- Jupyter Notebook
- Powershell Prompt (we'll talk about this more a little further into the course)

Learn more at https://www.anaconda.com/

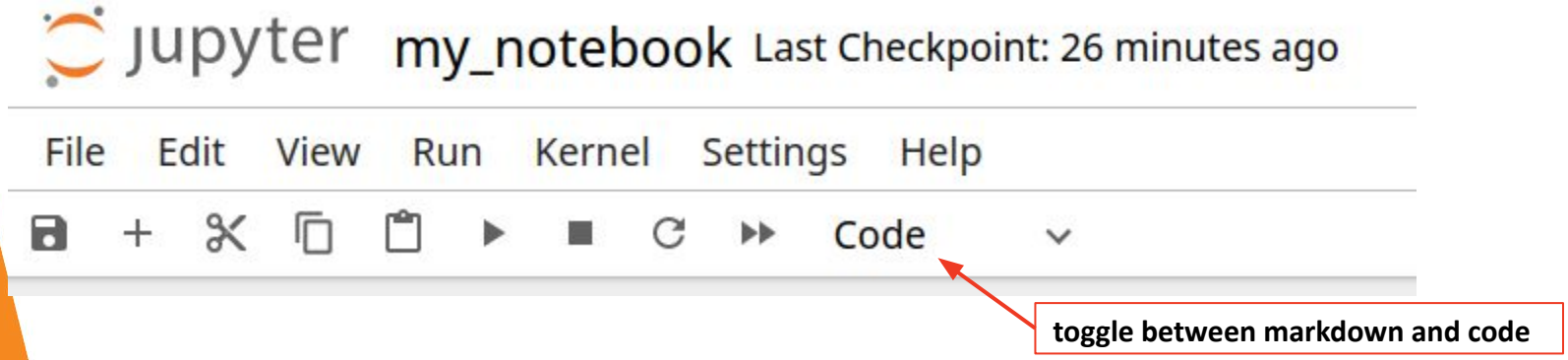
What is Jupyter Notebook?

Jupyter Notebook is an application that we will use to write and run Python code. It is organized with cells, providing an easy way to easily run just one particular component of code at a time.

The two main types of cells we will use are **code cells** and **markdown cells**.

- Code cells, as the name suggests, are where we will enter and run code.
- Markdown cells give us places to add text. The notebooks we go over in class have text for context/explanation, links to resources, etc. You can use markdown cells to add your own notes, flag things to come back to , etc. It is good practice to use markdown cells to help make your code more readable, both to others and to yourself in the future.
    - Markdown is a markup language, and learning a bit of syntax gives you the ability to add formatting to your notes.

How to designate cell type in Jupyter Notebook



File Path Review

You will see, and need to enter, file paths in your Python code to allow the notebook to find various files, primarily data files. This is an example of what those will look like:

`"../data/movies_data.csv"`

- The "`..`" component directs to move one file up in the hierarchy
- The "`/data`" directs to move into the data folder
- The "`/filename.fileextension`" directs to go into the file of interest, in this case it is the movies_data.csv

Keyboard Shortcuts for Jupyter Notebook

There are many keyboard shortcuts available when you're working in Jupyter Notebook. You can review a full list, but here are a few common ones:

- **Shift+Enter** - run the current cell and move to the next cell
- **a** - insert a cell above
- **b** - insert a cell below
- When clicked into a cell to type:
  - **Tab** - code completion, especially useful for completing variable names
  - **Ctrl+z** - undo changes, can click repeatedly to return to older version of that cell's code

Using Resources to Troubleshoot

Within Jupyter Notebook, there are a variety of ways to access documentation. We strongly encourage you to use them regularly as you begin to work on your own code.

You should also continue to build your search engine use skills. Typing “Python” (or a specific library) plus what you are trying to do or copying and pasting error messages into a search are good places to start. Pay attention to the sites you commonly see come up, and see which tend to have a format/style that works for you.

Dealing with Errors

Do not be afraid of getting an error in your code. Errors are useful and be be viewed as an opportunity to gain knowledge.

Read the output (stack trace) from the bottom up to see the error and what caused it

If you have no idea what is causing the error, try copying it, and pasting to a Google search.

```
In [2]: x + 2

---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-2-2c7a9192c558> in <module>()
----> 1 x + 2

NameError: name 'x' is not defined
```

Dealing with Errors

Often when you get an error, you will get a long stack trace.

Start by looking down here

```
In [34]: df.rename(lambda x: x.upper)

---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-34-18dc4f7001aa> in <module>()
----> 1 df.rename(lambda x: x.upper)

~/anaconda3/lib/python3.6/site-packages/pandas/util/_decorators.py in wrapper(*args, **kwargs)
    195         @wraps(func)
    196         def wrapper(*args, **kwargs):
--> 197             return func(*args, **kwargs)
    198
    199         if not PY2:

~/anaconda3/lib/python3.6/site-packages/pandas/core/frame.py in rename(self, *args, **kwargs)
   4023         kwargs.pop("axis", None)
   4024         kwargs.pop('mapper', None)
-> 4025         return super(DataFrame, self).rename(**kwargs)
   4026
   4027     @Substitution(**_shared_doc_kwargs)

~/anaconda3/lib/python3.6/site-packages/pandas/core/generic.py in rename(self, *args, **kwargs)
   1089                 level = self.axes[axis]._get_level_number(level)
   1090             result._data = result._data.rename_axis(f, axis=baxis, copy=copy,
-> 1091                                                     level=level)
   1092             result._clear_item_cache()
   1093

~/anaconda3/lib/python3.6/site-packages/pandas/core/internals/managers.py in rename_axis(self, mapper, axis, copy,
 level)
    169         """
    170         obj = self.copy(deep=copy)
--> 171         obj.set_axis(axis, _transform_index(self.axes[axis], mapper, level))
    172         return obj
    173

~/anaconda3/lib/python3.6/site-packages/pandas/core/internals/managers.py in _transform_index(index, func, level)
   2002         return MultiIndex.from_tuples(items, names=index.names)
   2003     else:
-> 2004         items = [func(x) for x in index]
   2005         return Index(items, name=index.name, tupleize_cols=False)
   2006

~/anaconda3/lib/python3.6/site-packages/pandas/core/internals/managers.py in <listcomp>(.0)
   2002         return MultiIndex.from_tuples(items, names=index.names)
   2003     else:
-> 2004         items = [func(x) for x in index]
   2005         return Index(items, name=index.name, tupleize_cols=False)
   2006

<ipython-input-34-18dc4f7001aa> in <lambda>(x)
----> 1 df.rename(lambda x: x.upper)

AttributeError: 'int' object has no attribute 'upper'
```

Stack Overflow

https://stackoverflow.com/

When viewing a question on Stack Overflow remember that the question is at the top and the answers are underneath.

Pay attention to upvotes and the selected answer.

Example:
https://stackoverflow.com/questions/11346283/renaming-columns-in-pandas

Package API Reference

https://pandas.pydata.org/pandas-docs/stable/reference/index.html

Reading the package documentation can be more difficult when you are first learning but as you develop your Python vocabulary and knowledge, you'll find that referring to them can be extremely helpful when writing or troubleshooting code.

Example:
https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.rename.html

Cheat Sheets

https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

https://www.datacamp.com/cheat-sheet/pandas-cheat-sheet-for-data-science-in-python